

Lec3-1

CO523 Programming Languages

DoCE

FoE

UoP



PART 4: EXECUTION MODELS



Compilation vs. Interpretation

Compiled: C, C++ → machine code

Interpreted: Python, Ruby → executed line by line

Hybrid: Java, C# → bytecode + VM

Virtual Machines

- JVM
- .NET Common Language Runtime
- Platform independence
- JIT compilation

Runtime Environment

- Stack
- Heap
- Static memory
- Execution model of a typical program

★ PART 5: LANGUAGE IMPLEMENTA TION DETAILS

– Scoping

- **Static/lexical scope** (C, Java, Python)
- **Dynamic scope** (early Lisp, Bash)

Lifetime of Variables

- Static
- Stack-dynamic
- Heap-dynamic

Memory Management

- Manual memory management (C)
 - malloc, free
 - Automatic memory management
 - Garbage collection (Java, Python)

Exception Handling

- Importance for reliability
- Checked vs. unchecked exceptions
- Examples in Java and Python

★ PART 6: MODERN LANGUAGE TRENDS

★ Popular Modern Languages and Why

- Python → AI & scripting
- Rust → safety + performance
- Go → concurrency
- TypeScript → safe web dev
- Kotlin → Android & JVM apps

– Safety and Security Features

- Memory-safe languages
- Type safety
- Null-safety
- Concurrency safety (Rust borrow checker)

– Future of Programming Languages

- More AI-assisted development
- Stronger static guarantees
- Safer concurrency models
- Domain-specific languages (DSLs)

– Summary

- we covered:
- Language concepts and evaluation
- Syntax, semantics, types
- Paradigms
- Scoping + runtime models
- Memory management
- Trends in modern languages

Questions?